



Bases de Datos

Conexión con bases de datos

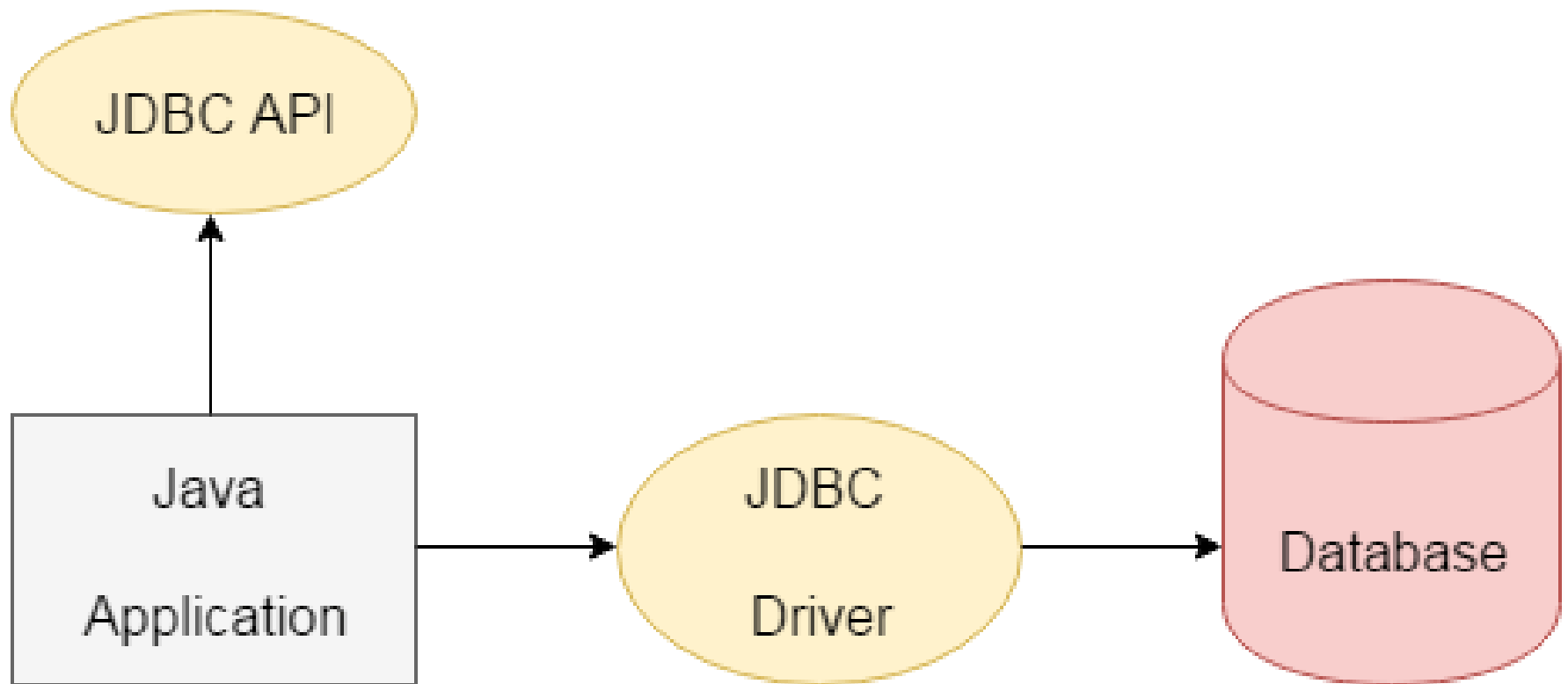
Susana Medina Gordillo
susana.medina@correounivalle.edu.co

Conexión con una base de datos

—

—

JDBC



JDBC



Los programas en Java se comunican con las bases de datos y manipulan sus datos utilizando la API **JDBC** (**Java Data Base Connectivity**)

La mayoría de los sistemas de administración de bases de datos (**MySQL, Oracle, Microsoft SQL Server, DB2, Informix**) incluyen controladores de **JDBC**.

Un **controlador de JDBC** implementa la **interfaz** para una base de datos específica (conjunto de clases que permite realizar la conexión y llevar a cabo la manipulación de datos desde una aplicación Java). La versión más reciente es 4.3 (Sept. 21/2017)

JDBC es equivalente a Open Database Connectivity (**ODBC**) de **Microsoft**.



Interfaces para conexión con JDBC

- ★ Driver interface
- ★ Connection interface
- ★ Statement interface
- ★ PreparedStatement interface
- ★ CallableStatement interface
- ★ ResultSet interface
- ★ ResultSetMetaData interface
- ★ DatabaseMetaData interface
- ★ RowSet interface



Clases para conexión con JDBC

- ★ DriverManager class
- ★ Blob class
- ★ Clob class
- ★ Types class



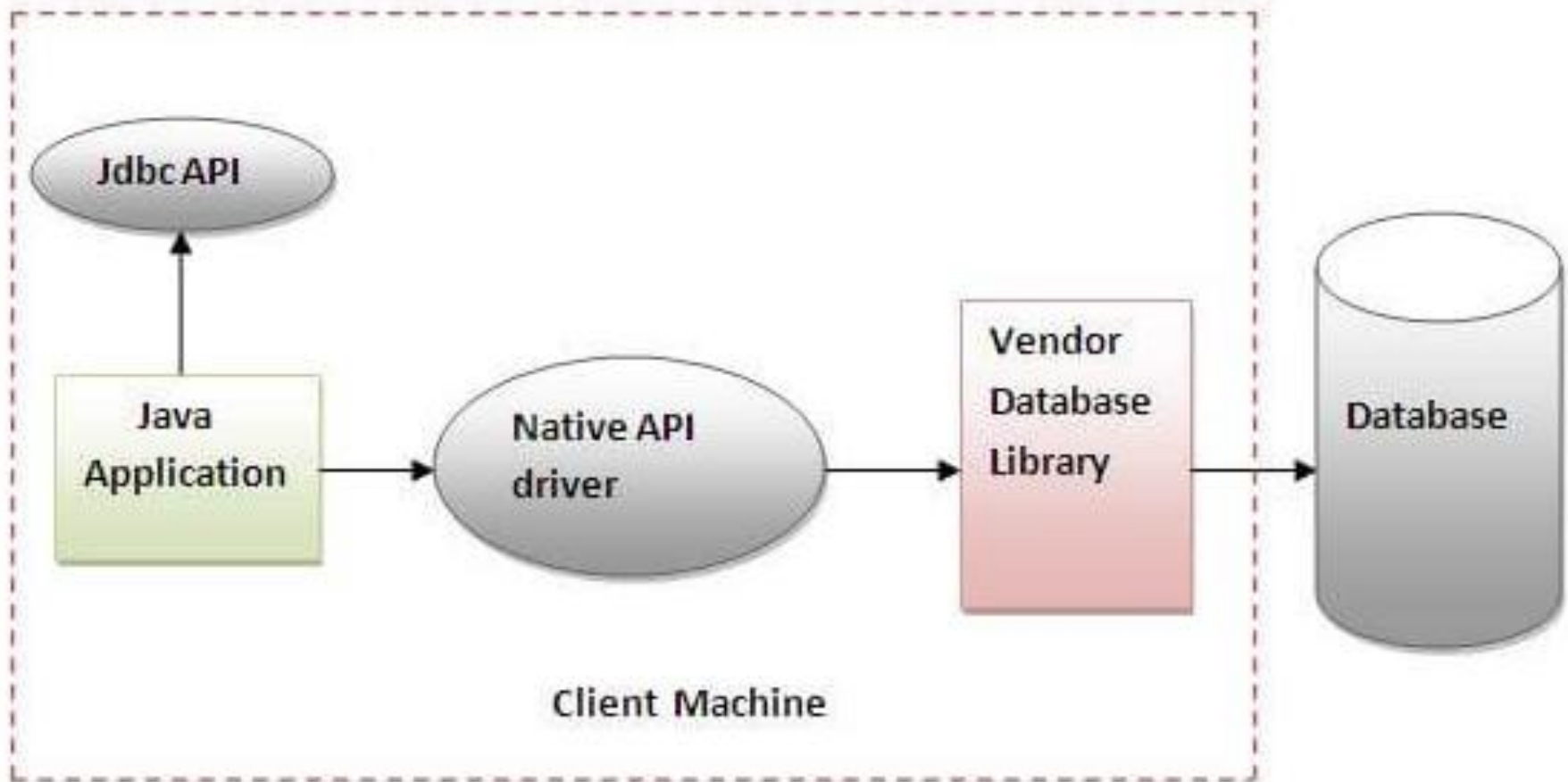
Actividades utilizando JDBC API

1. Conectarse a la base de datos
2. Ejecutar consultas (*queries*) y actualizar declaraciones (*statements*) de la base de datos.
3. Recuperar resultados (datos) recibidos de la base de datos.

Tipos de controladores (drivers)



1. JDBC-ODBC Bridge Driver (no nativo)
2. **Native Driver (nativo)**
3. Network Protocol Driver (middleware)
4. Thin Driver (directo: protocolo BD)



Controlador Nativo

Configuración del controlador



Descargar el archivo

db-connector-java-5.0.4-bin.tar.gz

Descomprimir el archivo

```
tar -zxvf db-connector-java-5.0.4-bin.tar.gz
```

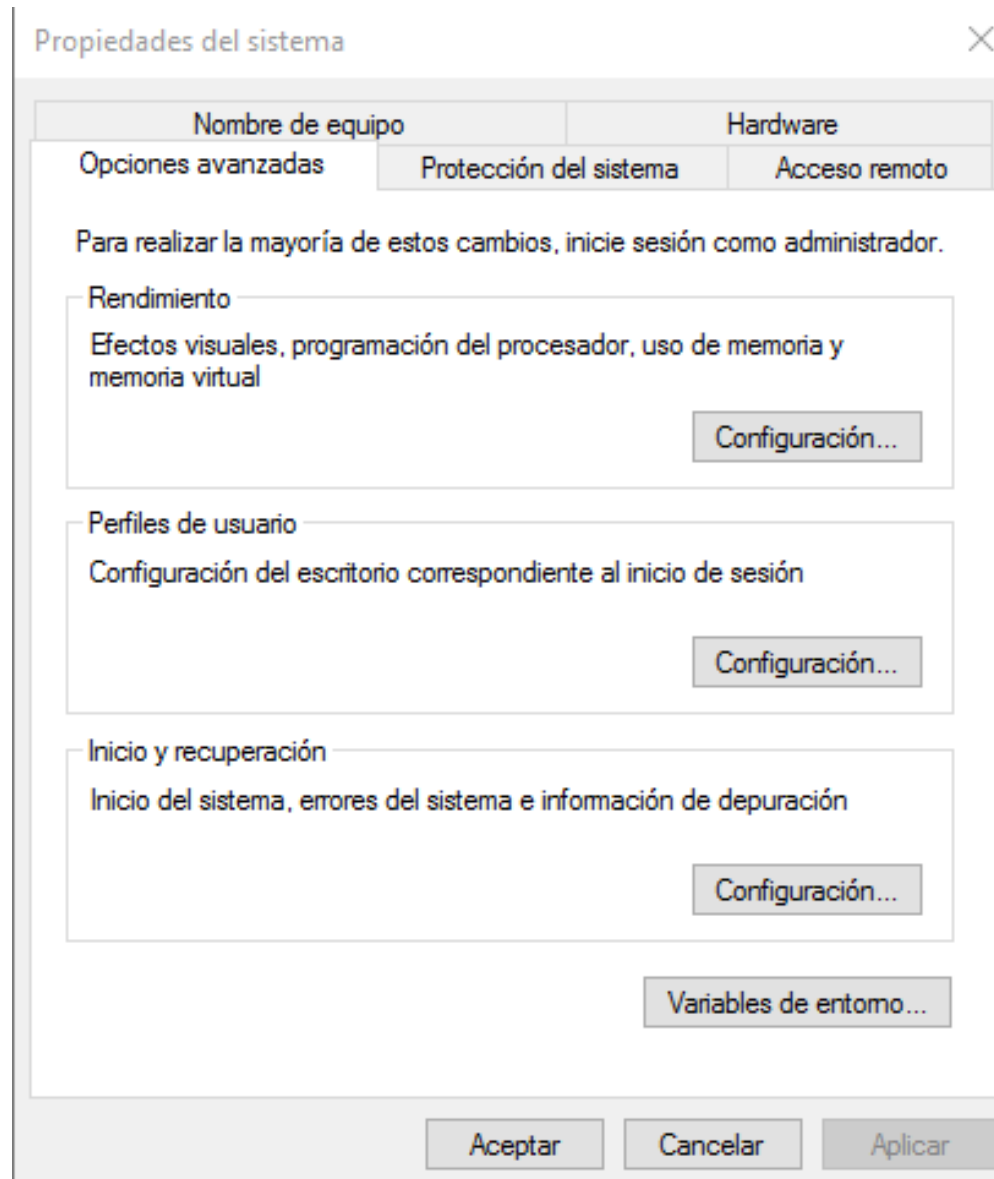
Copiar de la carpeta descomprimida al raíz, el archivo

bd-connector-java-5.0.4-bin.jar

Colocar el jar en el PATH

abrir configuración avanzada del sistema y añadir dirección a la variable PATH

PATH- Configuración avanzada del sistema



PATH- variables de entorno

Variables de entorno



Variables de usuario para susie

Variable	Valor
MOZ_PLUGIN_PATH	
OneDrive	C:\Users\susie\OneDrive
Path	%USERPROFILE%\AppData\Local\Microsoft\WindowsApps;
ProgramData	C:\ProgramData
TEMP	%USERPROFILE%\AppData\Local\Temp
TMP	%USERPROFILE%\AppData\Local\Temp

Nueva...

Editar...

Eliminar

Variables del sistema

Variable	Valor
ComSpec	C:\WINDOWS\system32\cmd.exe
NUMBER_OF_PROCESSORS	4
OS	Windows_NT
Path	C:\ProgramData\Oracle\Java\javapath;C:\Windows\system32;C:\Wi...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC;.PY
PROCESSOR_ARCHITECTURE	AMD64
PROCESSOR_IDENTIFIER	Intel64 Familly 6 Model 69 Stepping 1. GenuineIntel

Nueva...

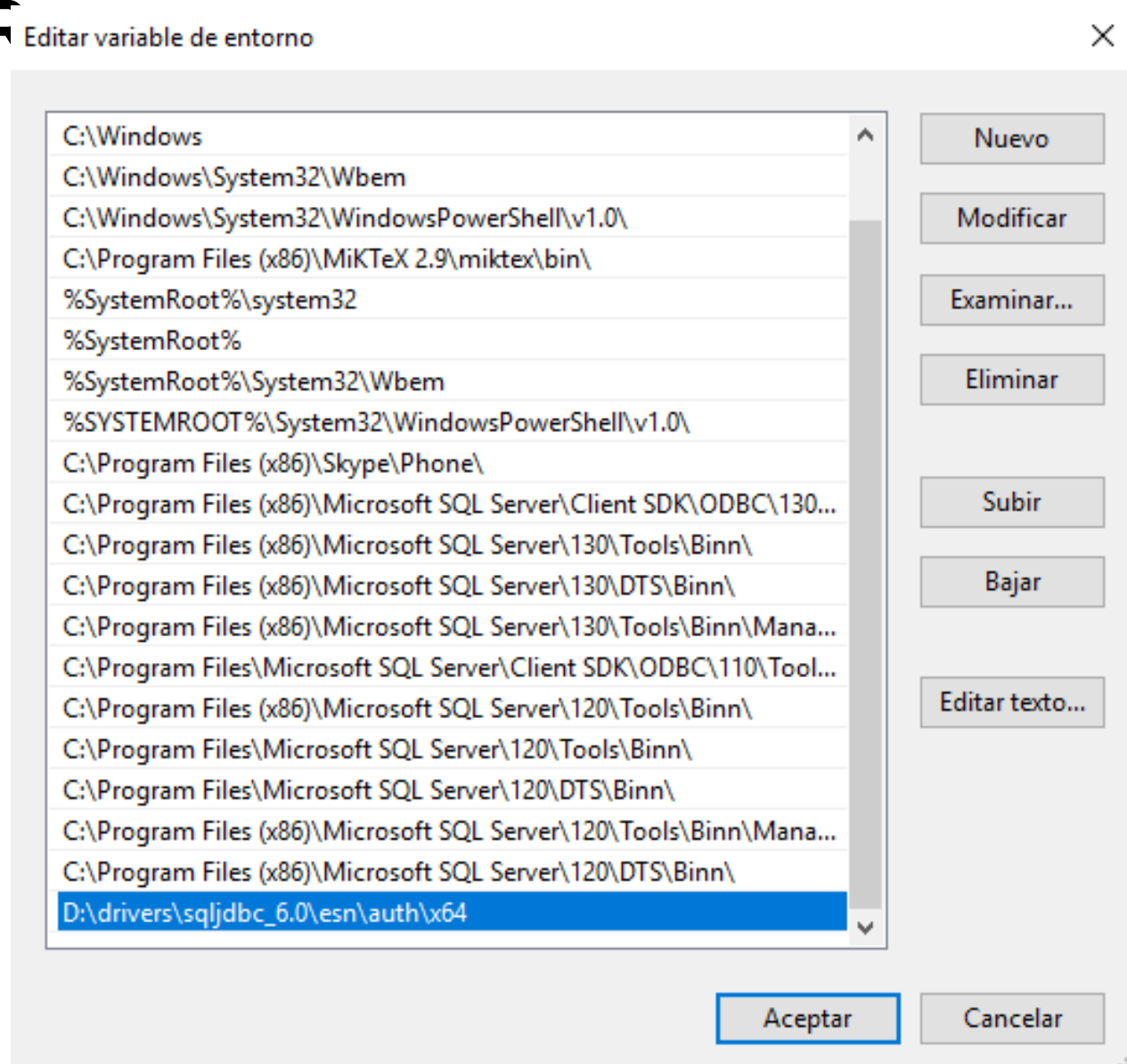
Editar...

Eliminar

Aceptar

Cancelar

PATH- agregando la dirección del .JAF



Formato general de la URL de conexión I



```
jdbc:sqlserver://[serverName[\instanceName]  
[:portNumber]  
[;property=value[;property=value]]
```

`jdbc:sqlserver://` (obligatorio) es el subprotocolo y es constante.

`serverName` (opcional) es la **dirección del servidor** con el que se establece la conexión. Puede ser un DNS o una dirección IP o bien un host local o `127.0.0.1` para el equipo local. Si no se especifica en la URL de conexión, es necesario especificar el nombre del servidor en la colección de propiedades.

Formato general de la URL de conexión II



```
jdbc:sqlserver://[serverName[\instanceName]  
[:portNumber]  
[;property=value[;property=value]]
```

instanceName (opcional) es la instancia para establecer la conexión con **serverName**. Si no se especifica, se establece una conexión con la instancia predeterminada.

portNumber (opcional) es el puerto para establecer la conexión con **serverName**. El valor predeterminado es **1433**. Si usa el valor predeterminado, no es necesario especificar el puerto ni el signo ":" precedente en la dirección URL.

Conexiones según la BD I



Conexión con la base de datos predeterminada del equipo local con autenticación integrada:

```
jdbc:sqlserver://localhost;integratedSecurity=true;
```

Conexión a la base de datos con nombre de un servidor remoto:

```
jdbc:sqlserver://localhost;databaseName=AdventureWorks;integratedSecurity=true;
```

Conexiones según la BD II



Conexión en el puerto predeterminado con el servidor remoto:

```
jdbc:sqlserver://localhost:1433;databaseName=AdventureWorks;integratedSecurity=true;
```

Conectar mediante la especificación de un nombre de aplicación personalizada:

```
jdbc:sqlserver://localhost;databaseName=AdventureWorks;integratedSecurity=true;applicationName=MyApp;
```



Pasos para conectarse a la BD

- 1) Registrar el **Driver class**
- 2) Crear la conexión (**Connection**)
- 3) Crear la declaración (**Statement**)
- 4) Ejecutar **queries** (consultas)
- 5) Cerrar la conexión

Conectarse a la BD desde un programa en Java I

Cargar el controlador de la base de datos. Este controlador está en el .jar que ya fue colocado en el PATH.

```
try {  
    Class.forName("oracle.jdbc.driver.OracleDriver").newInstance();  
}  
catch (ClassNotFoundException e) {  
    System.out.println("No encontró el driver");  
}
```



Sintaxis método getConnection

1. `public static Connection
getConnection(String url) throws SQLException`
2. `public static Connection
getConnection(String url, String name, String
password) throws SQLException`

Conectarse a la BD desde un programa en Java II

Crear un objeto Connection donde se especifica la ubicación de la base de datos, el login y el password

```
Connection conexion=null;
try
{
conexion = DriverManager.getConnection
("jdbc:oracle:thin:@localhost:1521:xe","system", "password");
} catch (Exception e) {
System.out.println( "No se pudo crear la
conexión" );
}
```



Enviando Instrucciones a la BD

Utilizar el método `createStatement` de **Connection** para obtener un objeto que implemente a la interfaz **Statement**. El programa utiliza al objeto para enviar instrucciones a la base de datos

```
Statement s = conexion.createStatement();
```




Ejecutando consultas en la BD

Utilizar el método `executeQuery` de `Statement` para enviar una consulta. Este método devuelve un objeto que implementa la interfaz `ResultSet` y que contiene el resultado de la consulta.

```
ResultSet rs = s.executeQuery ("SELECT *  
FROM Empleados");
```

Procesando consultas a la BD: posición



Procesar el objeto **rs** para visualizar la consulta:

```
while (rs.next()) {  
    area.append((String) rs.getObject(1));  
    area.append("\t" + (String) rs.getObject(2));  
    area.append("\t" + (String) rs.getObject(3));  
    area.append("\t" + (String) rs.getObject(4));  
    area.append("\t" + (Date) rs.getObject(5));  
    area.append("\t" + (Date) rs.getObject(6));  
    area.append("\n");  
}
```

Procesando consultas a la BD:

atributo

En el método **getString** se puede especificar el nombre del atributo.

```
while( rs.next() ) {  
    String nombre = rs.getString( "nombre" );  
    String propietario = rs.getString(  
        "propietario" );  
    String especie = rs.getString( "especie"  
    );  
    System.out.println( nombre + " " +  
        propietario + " " + especie);  
}
```

Procesando consultas a la BD: objetos



Se guarda el resultado en un arreglo de tipo Object. El índice de rs empieza en 1

```
while (rs.next())  
{  
Object[] datos = new Object[6];  
for (int i=0 ; i<6 ; i++ ){  
datos[i] = rs.getObjet(i+1);  
System.out.println(datos[i]);  
}  
}
```

Cerrando la conexión con la BD

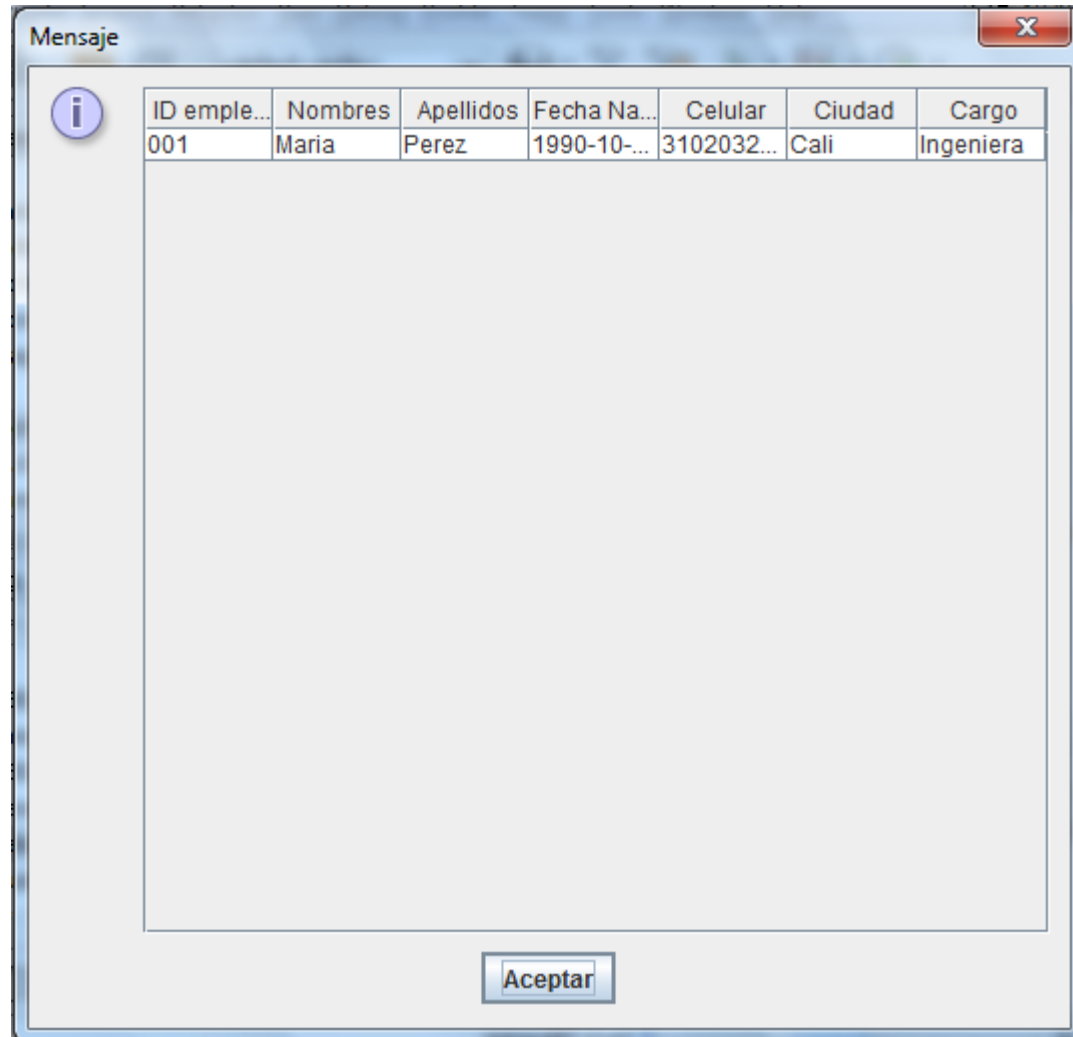


La conexión con la base de datos debe cerrarse con la instrucción:

```
conexion.close() ;
```

Todas las instrucciones deben estar en un **try-catch**.

Resultados de una consulta SQL con JTable



Receso 15 min



Práctica en Laboratorio